What Makes A Blockbuster Video Game?

Bryce Kratzer (1003433741), Sunny Jin (1006806306), Bryce Du (1005062557)

2023-12-02

Introduction – Bryce K.

We decided to use the videogames dataset, and attempt to answer the question "what makes a blockbuster videogame?" As group 11 we met and decided that Bryce Du would do the description of the dataset, Bryce Kratzer would be responsible for building the model and model validation, and Sunny Jin would be responsible for doing model diagnostics. Our research question "what makes a blockbuster videogame?" is quite a complicated one to answer, as not even market analysts are extremely accurate with their predictions. There have been many games that were expected to be massive but failed in the market. An example of one such game would be "Cyberpunk 2077" (2020) from CD Projekt RED, which before its launch had a tremendous amount of excitement from fans for its launch as well as everything pointing towards a big success. However, after launch it performed worse than expected and is now used as the template for a misleading game due to how unplayable it was. We hope to be able to provide a decent prediction model for the sales of videogames. Keeping in mind that we don't have a variable for how developed a game was, we hypothesize that the sales of a videogame can be predicted by a combination of rating, review score, used price, and publisher.

Data Cleaning – Bryce K.

Upon first inspecting the dataset provided to us, it is clear that there are erroneous entries in the dataset. For instance, "Call of Duty 2" was rated E while "Pokemon Dash" was rated T. Also, the "publisher" variable also included who published the game internationally, even though that wouldn't have affected US sales. Furthermore, the original dataset also featured several boolean variables which were solely "TRUE" for all entries. To resolve the ratings and boolean issues I merged the data with another dataset ("Managerial_and_Decision_Economics_2013.csv"). However, the second dataset wasn't without its own faults, as some entries didn't have a rating and it also had international publishers. For the games with no ratings, I set them to have a rating of E since many of them didn't have a rating on the ESRB (the ratings board for videogames in North America) website "https://www.esrb.org/search/". For the publisher's issue, I simply Googgled each game with two listed publishers and deleted the international publisher. We decided not to include all of the timed variables except Length.All.PlayStyles.Average,Length.All.PlayStyles.Leisure, Length.All.PlayStyles.Median, and Length.All.PlayStyles.Rushed. We did this because the timed variables were all highly correlated with each other and didn't add much value to the model.

Model Building – Bryce K.

To do any meaningful analysis we first need to split the data into training and testing datasets, and I opted to go with a 70/30 split. At first, I simply made the training set the first 70% of entries, and the testing set the last 30%. However, this resulted in poor performance and some variables having zero variance which caused a problem when I tried to scale the variables. In the end, I decided to use the script below to do a 70/30 split stratified by the year of release. I settled on using the release year as the stratification point because it made the most sense to me since the dataset should have a good diversity of release year. Once I had the training and testing sets complete, I used LASSO regression to make the model. I chose LASSO due to my experience using it in another course, where I learned that LASSO is a very powerful tool for feature

selection (especially with many variables like we have here). From there I did iteration experiments where I tried out different interactions between variables.

```
library(glmnet)
# Function to create model matrix
create_model_matrix = function(data) {
    model.matrix(~ . + Metadata.Genre.Action:Metadata.Genre.RPG
                 + Metadata.Genre.Action:Metadata.Genre.Strategy
                 + Metadata.Genre.Action:Metadata.Genre.Sports
                 + Metadata.Genre.Action:Metadata.Publisher.Activision
                 + Metadata.Genre.Action:Metadata.Publisher.Microsoft
                 + Metadata.Genre.Action:Metadata.Publisher.Rockstar
                 + Length.All.PlayStyles.Average:Metrics.Used.Price
                 + Length.All.PlayStyles.Leisure:Metrics.Used.Price:Metrics.Review.Score
                 + Metrics.Used.Price:Metrics.Review.Score
                 - Metrics.Sales - Title, data=data)
}
# Load the training and testing data
data = read.csv("~/transformed_alt_red_MAIN_categorical.csv")
training_data = read.csv("~/1-training-stratifed-alt-reduced.csv")
testing_data = read.csv("~/1-testing-stratifed-alt-reduced.csv")
# Response variable
y_train = training_data$Metrics.Sales
y_test = testing_data$Metrics.Sales
# Creating model matrices for training and testing datasets
X_train = create_model_matrix(training_data)
X_test = create_model_matrix(testing_data)
# Ensure that the predictors in the test set match the model matrix from the training set
X_test = X_test[, colnames(X_train)]
# Scale the predictor variables
# Important to use training set's mean and sd to scale the test set to prevent data leakage
X_train_scaled = scale(X_train[, -1])
X_test_scaled = scale(X_test[, -1], center = attr(X_train_scaled, "scaled:center"),
                      scale = attr(X_train_scaled, "scaled:scale"))
# Fit the LASSO model on training data
set.seed(2609)
cv_lasso = cv.glmnet(X_train_scaled, y_train, alpha = 1)
# Find the optimal lambda
lambda_optimal = cv_lasso$lambda.min
# Coefficients at the optimal lambda
coef_lasso = coef(cv_lasso, s = lambda_optimal)
# Print the coefficients
#print(coef_lasso)
```

Model Validation –Bryce K & Sunny Jin

For the model validation step I pretty much repeated the previous steps, but using the test set.

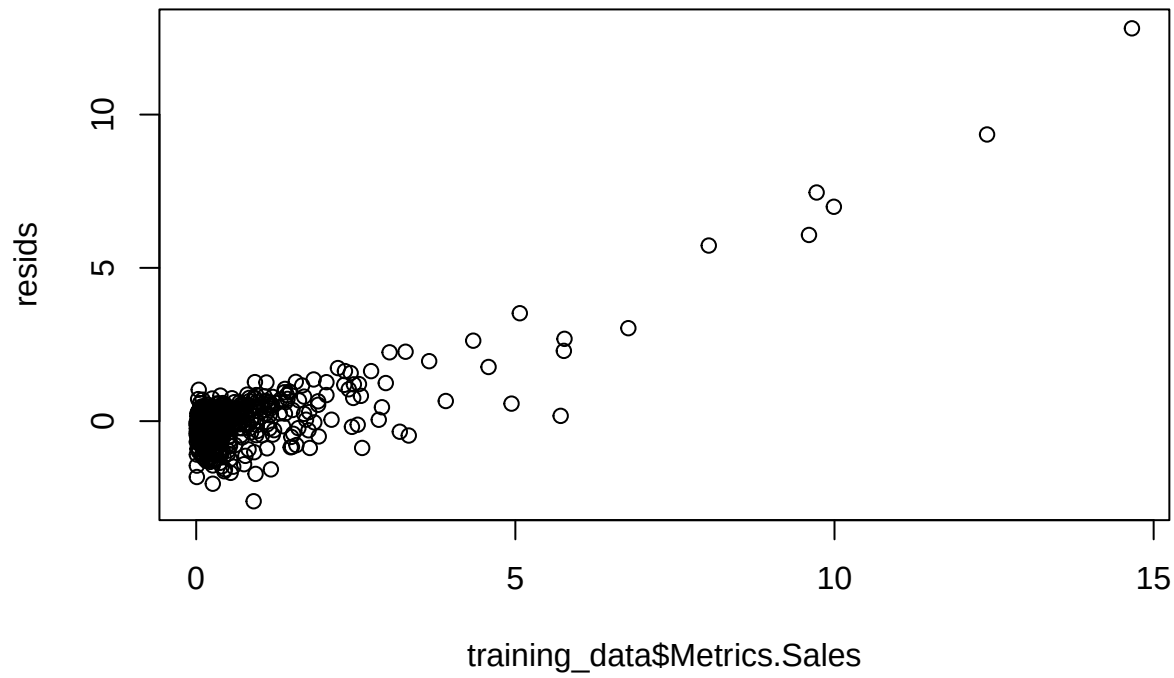
```
# Predict on test data
predictions = predict(cv_lasso, newx = X_test_scaled, s = "lambda.min")

# Calculate performance metrics
mse = mean((predictions - y_test) ^ 2)
rmse = sqrt(mse)
mae = mean(abs(predictions - y_test))
```

```r
ss_total = sum((y_test - mean(y_test)) ^ 2)
ss_residual = sum((y_test - predictions) ^ 2)
r_squared = 1 - (ss_residual / ss_total)
n = dim(training_data)[1]
p = length(coef_lasso)
adj_r_squared = 1 - (1 - r_squared) * ((n - 1) / (n - p - 1))
#Reduce to 4 significant digits
mse=round(mse,4)
rmse = round(rmse,4)
mae = round(mae,4)
r_squared = round(r_squared,4)
adj_r_squared = round(adj_r_squared,4)

#MSE, R^2, R_adj^2 of model including outliers:
c(mse,r_squared, adj_r_squared)
```

```
## [1] 0.4932 0.3532 0.3023
```

Model Diagnostics – Sunny Jin & Bryce D

```r
#coeff = read.csv("~/current best model coef.csv", header=TRUE)

designMatrix <- model.matrix(~ . + Metadata.Genre.Action:Metadata.Genre.RPG
                       + Metadata.Genre.Action:Metadata.Genre.Strategy
                       + Metadata.Genre.Action:Metadata.Genre.Sports
                       + Metadata.Genre.Action:Metadata.Publisher.Activision
                       + Metadata.Genre.Action:Metadata.Publisher.Microsoft
                       + Metadata.Genre.Action:Metadata.Publisher.Rockstar
                       + Length.All.PlayStyles.Average:Metrics.Used.Price
                       + Length.All.PlayStyles.Leisure:Metrics.Used.Price:Metrics.Review.Score
                       + Metrics.Used.Price:Metrics.Review.Score
                       - Metrics.Sales - Title, data=training_data)

coeff_value_matrix <- as.matrix(coef_lasso)

#E(Y) = Y~ = X * \beta
#predicted_vals <- designMatrix %*% coeff_value_matrix #This is insanely off
predicted_vals <- predict(cv_lasso, newx = X_train_scaled, s = "lambda.min")

#e = Y - Y~ (Y~ is predicted Y vector, E(Y))
resids <- training_data$Metrics.Sales - predicted_vals
```

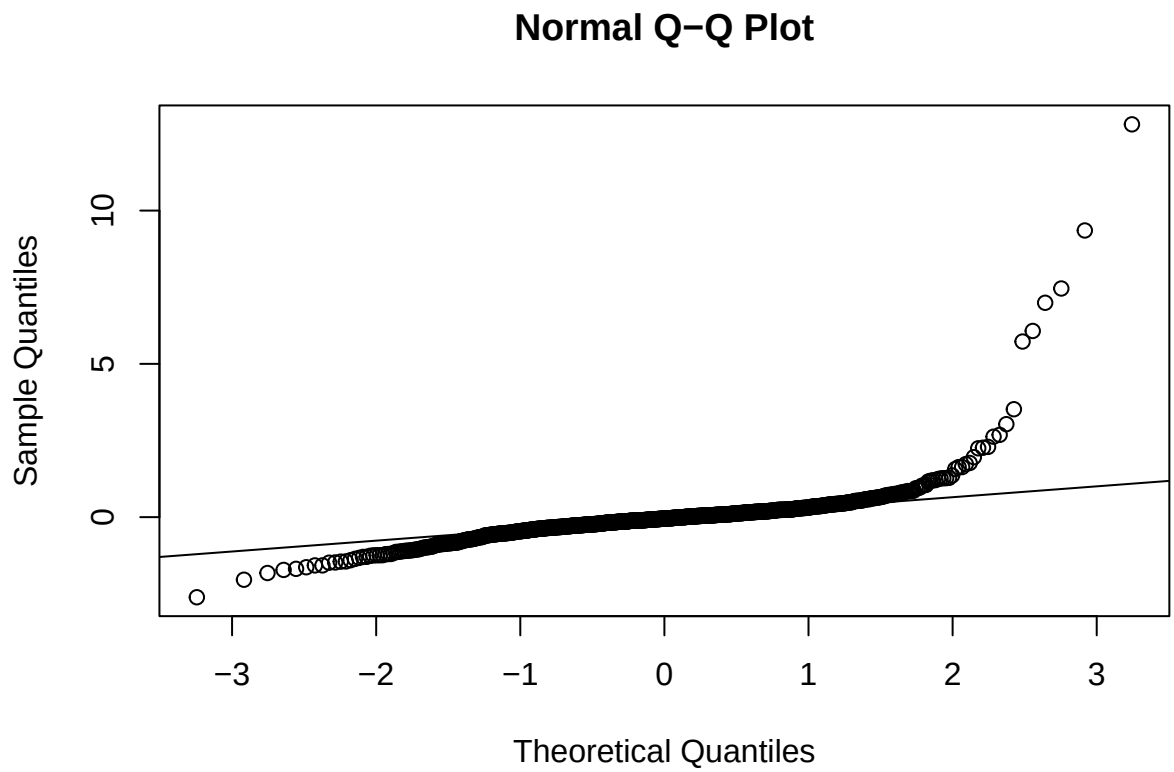


The residual plot shows no significant trend of a non-linear term. We find no evidence of the model being non-linear.

```
#Normality test:
shapiro.test(resids)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  resids
## W = 0.55772, p-value < 2.2e-16
```

```
qqnorm(resids)
qqline(resids)
```



The slope of the QQ line is not close to 1, so we may suspect normality assumption is violated. The p-value for Shapiro-Wilk's test is small, but since n is large, this test may not be entirely accurate.

```
#Test for Equal Variance (BF Test):
m = median(predicted_vals)
new_df = data.frame(predicted_vals, resids)
names(new_df) = c("predicted_vals", "resids")
resids.G1 = new_df[(new_df$predicted_vals <= m), ]
resids.G2 = new_df[(new_df$predicted_vals > m), ]
median.G1 = median(resids.G1$resids)
median.G2 = median(resids.G2$resids)

d1 = abs(resids.G1$resids - median.G1)
d2 = abs(resids.G2$resids - median.G2)
n1 = length(d1)
n2 = length(d2)
s1 = var(d1)
s2 = var(d2)

pprime = nrow(coeff_value_matrix)

s_sq = ((n1-1)*s1 + (n2-1)*s2 )/(n1+n2-pprime)
t = (mean(d1) - mean(d2))/sqrt(s_sq*(1/n1+1/n2))
BF = c(t, 2 * pt(t, n1 + n2 - pprime))
names(BF) = c("t", "p-value")
BF
```

```
##             t       p-value
```

```
## -6.749586e+00  2.874280e-11
```

Since p-value of BF test is extremely small, we reject H0 that the errors have equal variance. Thus, the variances between errors are not constant.

```r
#Remedial Measures: Box-Cox
lambda = BoxCox.lambda(training_data$Metrics.Sales, lower = -2, upper = 2)
lambda
```

```
## [1] -0.2380335
```

From the Box Cox Transformation, the lambda calculated by R is about -0.238. Further modeling will be done with the sales transformed by taking this power.

Remodeling With Transformation – Sunny Jin & Bryce K

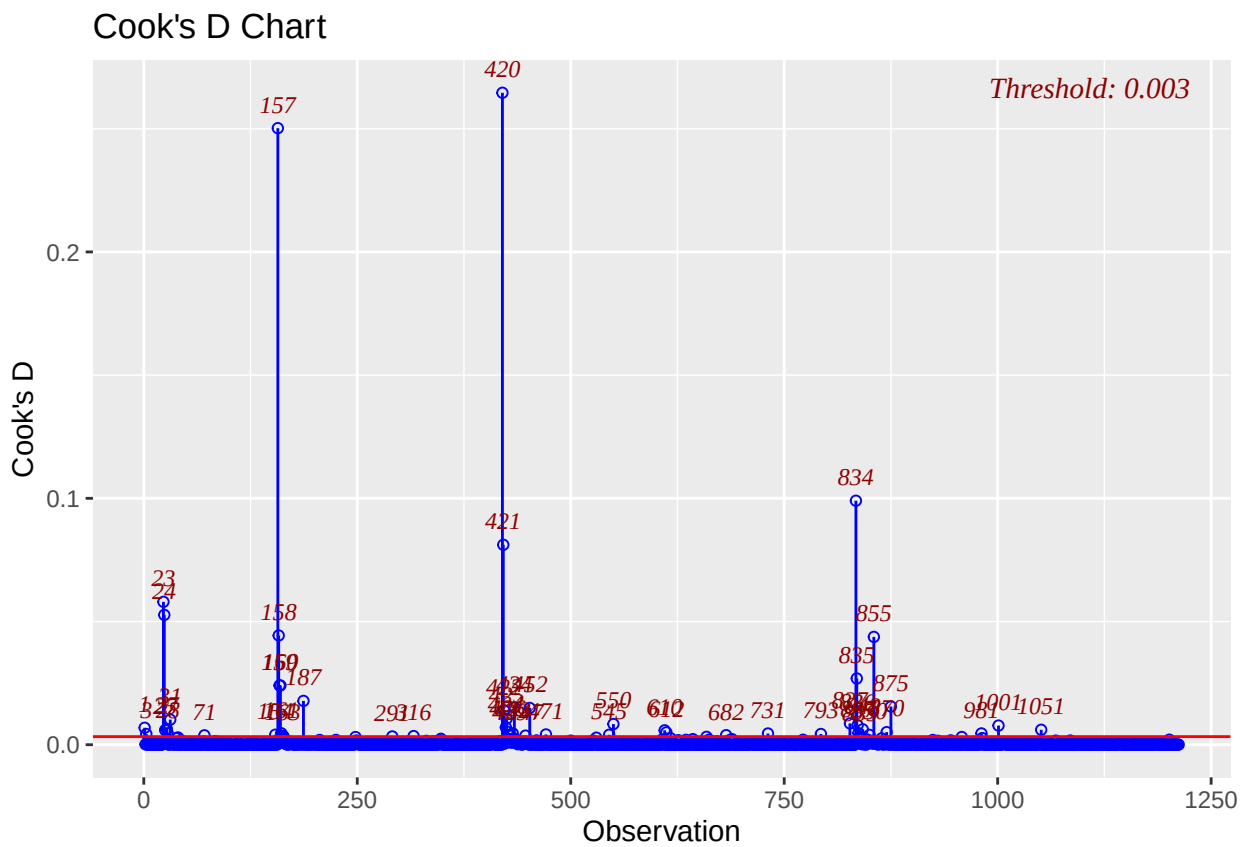
```r
Y=data$Metrics.Sales
predictors = create_model_matrix(data)
final_cv_lasso = cv.glmnet(predictors, Y, alpha = 1)
best_lambda = final_cv_lasso$lambda.min

final_lasso_model = glmnet(predictors, Y, alpha = 1, lambda = best_lambda)
lasso_coefficients = coef(final_cv_lasso, s = "lambda.min")[,1]
non_zero_coefficients = names(lasso_coefficients[lasso_coefficients != 0])

# Remove the intercept from the list if it's included
non_zero_coefficients = non_zero_coefficients[non_zero_coefficients != "(Intercept)"]

# Create a formula for the linear model using the non-zero coefficients
formula_lm = as.formula(paste("Metrics.Sales ~", paste(non_zero_coefficients, collapse = " + ")))

# Fit the linear model using the lm() function
final_lm_model = lm(formula_lm, data = data)

cooks_distances = cooks.distance(final_lm_model)

mse1 = mean(final_lm_model$residuals ^ 2)
# Calculate R-squared
ss_total1 = sum((Y - mean(Y)) ^ 2)
ss_residual1 = sum((Y - final_lm_model$fitted.values) ^ 2)
r_squared1 = 1 - (ss_residual1 / ss_total1)

n=dim(data)[1]
p = length(lasso_coefficients)
adj_r_squared1 = 1 - (1 - r_squared1) * ((n - 1) / (n - p - 1))

c(mse1, r_squared1, adj_r_squared1)
```

```
## [1] 0.7290584 0.3625078 0.3298584
```

Compared to the adjusted R^2 of 0.30 of the untransformed model, the adjusted R^2 of 0.33 for this model suggests an improvement.

Finding Outliers - Sunny Jin

```
#ols_plot_added_variable(final_lm_model)
p1 = ols_plot_cooksd_chart(final_lm_model)
```




```
print("Games with Cook's Distance greater than 0.04:")
```

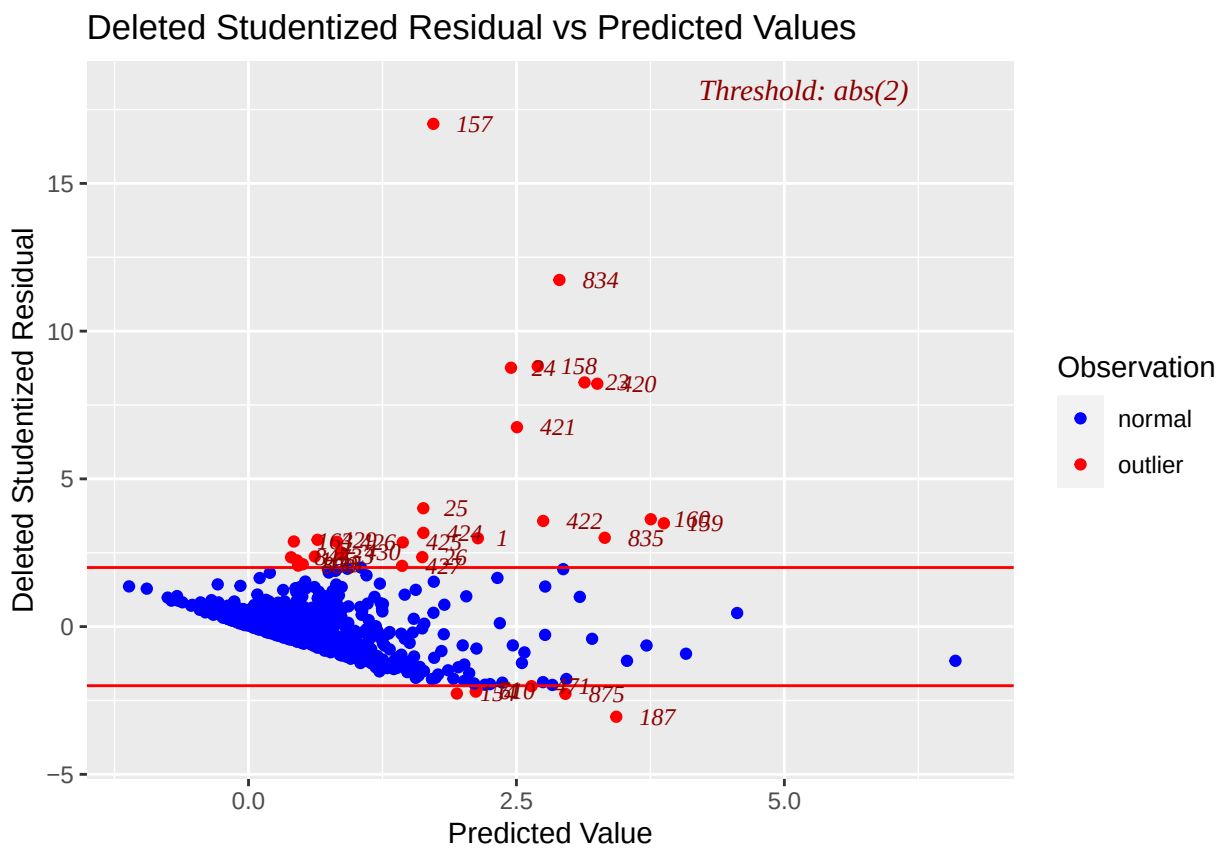
```
## [1] "Games with Cook's Distance greater than 0.04:"
```

```
rbind(which(cooks_distances > 0.04), data[which(cooks_distances > 0.04), 1])
```

```
##      23              24           157        158                      420
## [1,] "23"            "24"         "157"      "158"                    "420"
## [2,] "Mario Kart DS" "Nintendogs" "Wii Play" "New Super Mario Bros." "Wii Fit"
##      421      834              855
## [1,] "421"    "834"            "855"
## [2,] "Halo 3" "Mario Kart Wii" "Animal Crossing: City Folk"
```

Based on this output, there are 56 outliers. Out of which, No. 23 - Mario Kart DS, No. 157 - Wii Play, No. 158 - New Super Mario Bros., No. 420 - Wii Fit, No. 421 - Halo 3, No. 834 - Mario Kart Wii, No. 855 - Animal Crossing: City Folk have particularly large cook's distance (greater than 0.04), and thus particularly influential.

```
rstu = rstudent(final_lm_model)
rstd = rstandard(final_lm_model)
p5 = ols_plot_resid_stud_fit(final_lm_model)
```




```
stu_rejection_val = qt(1-0.05/(2*nrow(data)), nrow(data)-pprime-1)
outlying_stu_dis = which(rstu > stu_rejection_val)
print("Mean of studentized residual, and standardized residual:")
```

```
## [1] "Mean of studentized residual, and standardized residual:"
```

```
c(mean(rstu), mean(rstd))
```

```
## [1]  0.0026459654 -0.0004082761
```

```
print("Name, studentized resid and standardized resid of outliers:")
```

```
## [1] "Name, studentized resid and standardized resid of outliers:"
```

```
cbind(data[outlying_stu_dis, 1], rstu[outlying_stu_dis], rstd[outlying_stu_dis])
```

```
##     [,1]                    [,2]               [,3]
## 23  "Mario Kart DS"         "8.26421158612138" "8.03481805490708"
## 24  "Nintendogs"            "8.76118764325512" "8.48883521700519"
## 157 "Wii Play"              "17.0141868638953" "15.2285308489791"
## 158 "New Super Mario Bros." "8.81010567841219" "8.53327015182956"
## 420 "Wii Fit"               "8.22018282806104" "7.99437174401832"
## 421 "Halo 3"                "6.74844644637367" "6.62270990941566"
## 834 "Mario Kart Wii"        "11.7325081878345" "11.0980730144823"
```

This list matches the particularly outlying games with particularly large Cook's Distance, minus No. 855. Animal Crossing: City Folk. It is worth noticing that out of these 7 outliers, 6 are published by Nintendo. This may explain why Nintendo was the most influential game publisher, with the greatest coefficient. The final model will be calculated after removing the 7 outliers.

Final Model code

```
#which(colnames(data)==="")
#Re-run a model excluding outliers above:
data_no_outlier = data[-c(23,24,157,158,420,421,834), ]
Y2 = data_no_outlier$Metrics.Sales
predictors2 = create_model_matrix(data_no_outlier)
final_cv_lasso2 = cv.glmnet(predictors2, Y2, alpha = 1)
best_lambda2 = final_cv_lasso2$lambda.min

final_lasso_model2 = glmnet(predictors2, Y2, alpha = 1, lambda = best_lambda2)
lasso_coefficients2 = coef(final_cv_lasso2, s = "lambda.min")[,1]
#print(lasso_coefficients2)
non_zero_coefficients2 = names(lasso_coefficients2[lasso_coefficients2 != 0])

# Remove the intercept from the list if it's included
non_zero_coefficients2 = non_zero_coefficients2[non_zero_coefficients2 != "(Intercept)"]

# Create a formula for the linear model using the non-zero coefficients
formula_lm2 = as.formula(paste("Metrics.Sales ~", paste(non_zero_coefficients2, collapse = " + ")))

# Fit the linear model using the lm() function
final_lm_model2 = lm(formula_lm, data = data_no_outlier)

mse2 = mean(final_lm_model2$residuals ^ 2)
# Calculate R-squared
ss_total2 = sum((Y2 - mean(Y2)) ^ 2)
ss_residual2 = sum((Y2 - final_lm_model2$fitted.values) ^ 2)
r_squared2 = 1 - (ss_residual2 / ss_total2)

n_2=dim(data_no_outlier)[1]         #n2 was used in BF test
#p2 = length(lasso_coefficients2)  #Same as p
adj_r_squared2 = 1 - (1 - r_squared2) * ((n_2 - 1) / (n_2 - p - 1))

print("MSE, R^2, R_adj^2 of model including outliers:")
```

```
## [1] "MSE, R^2, R_adj^2 of model including outliers:"
```

```
c(mse1, r_squared1, adj_r_squared1)
```

```
## [1] 0.7290584 0.3625078 0.3298584
```

```
print("MSE, R^2, R_adj^2 of model excluding outliers:")
```

```
## [1] "MSE, R^2, R_adj^2 of model excluding outliers:"
```

```
c(mse2, r_squared2, adj_r_squared2)
```

```
## [1] 0.2827101 0.4635900 0.4359496
```

The adjusted R^2 of the model removing the outliers is around 0.44, which is significantly better than the adjusted R^2 of the lasso model without removing the outliers.

Conclusion

Final Model –Bryce K

Sales=0.6154-0.0621(Metrics.Used.Price)-0.0073(Metrics.Review.Score)+0.0312(Features.Max.Players)+0.0463(Features.Multip
0.0569(Release.Re.release)+0.0109(Length.All.PlayStyles.Average)-0.0051(Length.All.PlayStyles.Leisure)+0.0054(Length.All.Pl
0.0108(Length.All.PlayStyles.Rushed)+0.0205(Metadata.Genre.Action)-0.1887(Metadata.Genre.Adventure)+0.1588(Metadata.
0.0062(Metadata.Genre.Racing..Driving)-0.0058(Metadata.Genre.RPG)-0.0209(Metadata.Genre.Simulation)-
0.1374(Metadata.Genre.Sports)-0.2686(Metadata.Genre.Strategy)-0.0440(Metadata.Publisher.2K)+0.0919(Metadata.Publisher
0.1178(Metadata.Publisher.Atari)-0.0136(Metadata.Publisher.Capcom)+0.0783(Metadata.Publisher.Disney)+0.1226(Metadata
0.1783(Metadata.Publisher.Eidos)-0.0769(Metadata.Publisher.Konami)+0.2853(Metadata.Publisher.Microsoft)-
0.1775(Metadata.Publisher.Namco)+0.5065(Metadata.Publisher.Nintendo)-0.0206(Metadata.Publisher.Other)+0.5115(Metada
0.1761(Metadata.Publisher.SquareEnix)-0.0112(Metadata.Publisher.THQ)+0.0375(Release.Console.Nintendo.Wii)-
0.1261(Release.Console.PlayStation.3)-0.0943(Release.Console.Sony.PSP)+0.0732(Release.Console.X360)+0.0060(Release.Ratin
0.0901(Release.Rating_T)-0.1644(Metadata.Genre.Action:Metadata.Genre.RPG)+0.1752(Metadata.Genre.Action:Metadata.Ge

Interpretation –Bryce K

A notable trend in the model is the inverse relationship between both the used price (Metrics.Used.Price) and review scores (Metrics.Review.Score) with sales. Specifically, the model suggests that an increase in the used price of a game is associated with a modest decline in sales, with a coefficient of -0.0621. This could be reflective of market sensitivity to pricing among used games. Similarly, the negative coefficient for review scores, albeit small at -0.0073, hints at a counterintuitive relationship where higher review scores could correlate with slightly lower sales. This may suggest that other factors, such as genre or publisher reputation, could have a more significant influence on sales than the review scores themselves. Conversely, features such as the maximum number of players a game supports (Features.Max.Players) and whether a game is multiplatform (Features.Multiplatform) show a positive association with sales. The coefficients for these variables, 0.0312 and 0.0463 respectively, indicate that games designed for more players and those available across multiple gaming platforms tend to have higher sales. This aligns with the current market trend where social and accessible gaming experiences are highly valued. Additionally, the model incorporates interaction terms, such as the one between games classified as both Action and RPG genres. The positive coefficient for this interaction (0.1641) suggests that games that can be categorized within these two genres simultaneously are likely to see a boost in sales, which may be attributable to the broader appeal that such games carry. Furthermore, the brand power of publishers and the popularity of consoles play a significant role in sales. For instance, games published by Nintendo are associated with an increase in sales, as indicated by the coefficient of 0.5065, underscoring Nintendo's strong market presence. The positive coefficient for the Xbox 360 console (0.0732) also highlights its impact on game sales during the period under study.